

EECS 348 Assignment 1

GenAI Code Comparison

Name: Prabesh Sapkota

Date: September 8, 2026

1. GenAI Programs

GenAI 1: GitHub Copilot

I accessed GitHub Copilot on my computer.

Prompt used:

Write a C program that picks a fixed secret number between 1 and 10. The user gets up to 3 tries to guess the number. Use printf to prompt the user and scanf to read their guess. Use a while or for loop to let the user guess up to 3 times. Use if-else statements to check if the guess is too high, too low, or correct. Stop the loop immediately if the user guesses the correct number. Print a final win or lose message.

Code generated by Copilot:

```
#include <stdio.h>

int main() {
    int secret = 7;
    int guess;
    int attempt = 1;
    int max_attempts = 3;

    printf("Guess a number between 1 and 10.\n");

    while (attempt <= max_attempts) {
        printf("Attempt %d/%d. Enter your guess: ", attempt, max_attempts);
        scanf("%d", &guess);

        if (guess == secret) {
            printf("Correct! You win!\n");
            return 0;
        } else if (guess < secret) {
            printf("Too low! Try again.\n");
        } else {
            printf("Too high! Try again.\n");
        }

        attempt++;
    }

    printf("Sorry, you lose! The correct number was %d.\n", secret);

    return 0;
}
```

2. GenAI 2: Google Gemini

I accessed Google Gemini. I used the same prompt as Copilot.

Code generated by Gemini:

```
#include <stdio.h>

int main(void) {
    const int secret_number = 7;
    const int max_attempts = 3;
    int guess;
    int won = 0;

    printf("Guess a number between 1 and 10.\n");

    for (int attempt = 1; attempt <= max_attempts; attempt++) {
```

```
printf("Attempt %d/%d. Enter your guess: ", attempt, max_attempts);
scanf("%d", &guess);

if (guess == secret_number) {
    printf("Correct! You win!\n");
    won = 1;
    break;
} else if (guess < secret_number) {
    if (attempt < max_attempts) {
        printf("Too low! Try again.\n");
    } else {
        printf("Too low!\n");
    }
} else {
    if (attempt < max_attempts) {
        printf("Too high! Try again.\n");
    } else {
        printf("Too high!\n");
    }
}

if (!won) {
    printf("Game over! You ran out of tries. The number was %d.\n", secret_number);
}

return 0;
}
```

3. Comparison

Correctness: Both programs correctly meet the main requirements. They use the fixed number 7, allow up to three guesses, use printf and scanf, give high/low feedback, and stop when the correct number is guessed.

Execution time: Both programs are $O(1)$. The loop can only run three times, so the amount of work is limited by a constant.

Space complexity: Both programs are $O(1)$ because they only use a fixed number of variables and do not need additional memory as the number of attempts changes.

Maintainability: Both programs are fairly easy to understand. The Copilot version is shorter and more direct. The Gemini version uses a for loop and a separate won variable, which makes the final result explicit but adds some extra code.

4. Program Selected

I chose the Copilot program as the starting point. I chose it because it is simple and the structure is easy to follow. The while loop and if-else statements directly match the requirements. Since both programs have the same $O(1)$ time and $O(1)$ space complexity, I preferred the simpler version.

5. Improvements

Correctness: I changed the secret number and maximum attempts to constants so they cannot accidentally be changed. I also checked the return value of scanf so invalid non-integer input is detected.

Execution time: The program is still $O(1)$. The input check only adds a constant amount of work and the program still allows a maximum of three attempts.

Space complexity: The program is still $O(1)$. The extra won variable uses a constant amount of memory.

Maintainability: I changed main to main(void), used descriptive constants, and added comments to make the code easier to understand. I also used a won variable so the program can tell whether the user guessed correctly before printing the final losing message.

6. Final Program

```
/*
 * EECS 348 Assignment 1
 * Program: Number Guessing Game
 * Description: The program has a fixed secret number between 1 and 10.
 *              The user gets up to 3 tries to guess it.
 * Input: User enters an integer guess.
 * Output: Messages telling the user if the guess is high, low, correct,
 *         and a final win or lose message.
 * Collaborators: None
 * Other sources: GitHub Copilot and Google Gemini
 * Author: Prabesh Sapkota
 * Creation Date: September 8, 2026
 * Revision Date: September 8, 2026
 * Revisions: Added input checking, constants, and comments.
 */

#include <stdio.h>

int main(void) {
    // Fixed values for the game.
    const int secret = 7;
    const int max_attempts = 3;

    // Variables for the user's guess and game result.
    int guess;
    int won = 0;

    // Display the instructions.
    printf("Guess a number between 1 and 10.\n");

    // Give the user up to three attempts.
    for (int attempt = 1; attempt <= max_attempts; attempt++) {
        // Ask the user for a guess.
        printf("Attempt %d/%d. Enter your guess: ", attempt, max_attempts);

        // Read the guess and make sure it is an integer.
        if (scanf("%d", &guess) != 1) {
            printf("Invalid input. Please enter an integer.\n");
            return 1;
        }

        // Check if the guess is correct, low, or high.
        if (guess == secret) {
            printf("Correct! You win!\n");
            won = 1;
            break;
        } else if (guess < secret) {
            printf("Too low!");
            if (attempt < max_attempts) {
                printf(" Try again.");
            }
            printf("\n");
        } else {
            printf("Too high!");
            if (attempt < max_attempts) {
                printf(" Try again.");
            }
            printf("\n");
        }
    }

    // Print the losing message if the user did not guess correctly.
    if (!won) {
        printf("Sorry, you lose! The correct number was %d.\n", secret);
    }

    return 0;
}
```